

运动会报名系统：专家实施指南与设计文档

本指南旨在为“程序设计课程实践”中的“运动会报名系统”项目 1 提供一个全面的设计蓝图和分步实施策略。其目标是协助开发一个功能完整、结构清晰、符合课程要求的C语言应用程序。

I. 系统蓝图：运动会报名系统设计构想

坚实的设计是成功实施软件项目的前提。本章节详细阐述系统的核心需求、数据结构、模块划分及数据持久化策略，为后续的编码工作奠定基础。这直接关系到评分标准中对“模块化设计”的要求 1。

A. 解构任务：运动会报名系统的核心需求

此项目的核心目标是使用C语言开发一个运动会报名管理系统，该系统需有效管理运动项目、学生信息以及报名流程 1。

- 关键约束条件：**
 - 编程语言：C语言 1。
 - 团队规模背景：建议2-3人完成，暗示了项目的复杂程度适合小型团队协作 1。个人开发者需要系统地覆盖所有这些方面。
- 核心功能领域** (源自 1，亦在 1 中总结):
 - 运动项目管理
 - 学生信息管理
 - 项目报名管理
 - 统计查询功能
 - 其他功能 (登录/退出)

任务书明确指出：“相关小组开发的系统可以包含但不限于上述功能,需要自行设计更具有实用性和创新性的系统功能” 1。这表明，除了完成列表中的基础功能外，系统还鼓励开发者主动思考并增加实用价值。评分标准中也明确了对“难度及创新”的考量 1。因此，本指南在覆盖所有强制性功能的同时，也会提示潜在的创新方向。

B. 数据基石：定义核心C语言数据结构

根据任务书要求，程序必须使用结构体 (struct) 来存储各类信息 1。

- 1. 运动项目 (SportEvent) 结构体：**
 - 成员 (源自 1)

```
1  
;  
1  
);
```
 - 项目编号 (event_id): char event_id; (建议使用字符串以便灵活编号，例如 "EVT001")
 - 项目名称 (event_name): char event_name;
 - 项目类型 (event_type): char event_type; (例如: "田赛", "径赛")

- 参赛类型 (participation_type): `char participation_type;` (例如: "男子项目", "女子项目"。可考虑增加 "混合项目" 作为创新点)
- 比赛时间 (competition_time): `char competition_time;` (例如: "YYYY-MM-DD HH:MM"。若需进行时间计算, 可考虑 `struct tm`, 但在C控制台应用中, 字符串通常更易于直接输入输出)
- 比赛地点 (venue): `char venue;`
- 项目状态 (status): `char status;` (例如: "报名中", "已结束", "已取消")
- 报名人数上限 (registration_limit): `int registration_limit;`
- 当前报名人数 (current_registrations): `int current_registrations;` (此为派生/辅助字段, 对于检查 报名人数上限 至关重要)

• 2. 学生信息 (StudentInfo) 结构体:

◦ 成员 (源自

```
1
;
1
):
```

- 学号 (student_id): `char student_id;`
- 姓名 (name): `char name;`
- 性别 (gender): `char gender;` (例如: "男", "女")
- 班级 (class_info): `char class_info;`
- 学院 (college): `char college;`
- 联系方式 (contact_info): `char contact_info;` (例如: 电话或邮箱)

• 3. 报名信息 (RegistrationInfo) 结构体:

◦ 成员 (源自

```
1
;
1
):
```

- 报名编号 (registration_id): `char registration_id;` (例如: "REG0001", 可考虑自动生成)
- 学号 (student_id): `char student_id;` (关联学生信息的外键)
- 项目编号 (event_id): `char event_id;` (关联运动项目的外键)
- 报名时间 (registration_time): `char registration_time;` (记录报名发生的精确时间戳)
- 成绩 (score): `char score;` (可为空, 例如: "N/A", "10.5s", "5.50m")

• 数据存储方式选择: 数组与链表 1

◦ 数组

- ：
- 优点：实现简单，可通过索引直接访问（若已排序或ID映射到索引）。
 - 缺点：大小固定（增长时需动态重分配，可能复杂或低效），插入/删除可能较慢（需移动元素）。
- 链表
- ：
- 优点：大小动态（易于扩展/收缩），高效的插入/删除（找到节点后）。
 - 缺点：顺序访问（除非优化，否则搜索较慢），需要更多内存（用于指针），实现相对复杂。
- **项目建议：**对于此项目，若能合理预估最大记录数，从**动态分配的数组**入手较为简单。如果确实需要真正的动态伸缩能力和频繁的在数据集 中间进行插入/删除操作，则链表更为优越。对于本课程实践项目，动态数组（按需重新分配内存）能在简易性和功能性之间取得良好平衡。本指南将主要基于动态数组进行阐述，但会提及链表作为替代方案。

数据的内在联系和完整性是系统设计中不可忽视的一环。`报名信息` 结构通过ID将 `学生信息` 和 `运动项目` 联系起来。这意味着在一个区域的操作可能会影响其他区域（例如，删除一个学生应妥善处理其相关的报名记录）。运动项目的 `报名人数上限` 需要与该项目的当前报名人数进行实时核对。因此，系统设计必须考虑到这些关联。例如，当一个学生报名时，所选项目的 `当前报名人数` 必须增加；取消报名时则相应减少。用于根据ID检索学生或项目信息的辅助函数将是必不可少的。

表1: 数据结构定义

结构体名称	成员名称	建议C类型	描述 / 约束
SportEvent	event_id	char	项目编号
	event_name	char	100米、跳高、跳远等
	event_type	char	田赛、径赛等
	participation_type	char	男子项目、女子项目
	competition_time	char	比赛时间 (例如: "YYYY-MM-DD HH:MM")
	venue	char	比赛地点
	status	char	报名中、已结束
	registration_limit	int	报名人数上限
StudentInfo	current_registrations	int	<i>辅助字段:</i> 当前报名人数
	student_id	char	学号
	name	char	姓名
	gender	char	性别
	class_info	char	班级
	college	char	学院

结构体名称	成员名称	建议C类型	描述 / 约束
	contact_info	char	联系方式
RegistrationInfo	registration_id	char	报名编号 (可考虑自动生成)
	student_id	char	学号 (关联 StudentInfo)
	event_id	char	项目编号 (关联 SportEvent)
	registration_time	char	报名时间 (例如: "YYYY-MM-DD HH:MM:SS")
	score	char	成绩 (可为空)

此表格是数据模型的权威模式定义。11 列出了成员，而此表将其转化为具体的C类型，并增加了如字符串长度或辅助字段等必要细节。这确保了所有操作此数据的模块之间的一致性，从一开始就明确这些定义可以防止编码过程中的错误和误解，直接有助于构建一个更稳健的系统，是将需求转化为可实现设计的第一步。

C. 架构规划：模块化设计以提升清晰度与可维护性

模块化的重要性在评分标准的“基本功能”部分得到了强调：“能按照模块化设计要求, 进行功能模块的划分, 每个功能模块的粒度合适、功能相对独立” 1。模块化方法使代码更易于编写、测试、调试和维护。

- 建议的C模块 (源文件和头文件)
：
 - `event_management.c` / `event_management.h`: 负责运动项目CRUD（创建、读取、更新、删除）及搜索功能。
 - `student_management.c` / `student_management.h`: 负责学生信息CRUD及搜索功能。
 - `registration_management.c` / `registration_management.h`: 负责管理报名相关操作（学生报名、取消报名、成绩录入）。
 - `statistics_reporting.c` / `statistics_reporting.h`: 负责所有统计查询功能及将其保存到文件。
 - `file_io.c` / `file_io.h`: 通用的文件加载与保存功能，用于处理主要数据（项目、学生、报名信息），集中化文件操作。
 - `utils.c` / `utils.h`: 工具函数（例如：输入验证、字符串处理、ID生成（如果需要）、日期/时间辅助函数）。
 - `main.c`: 包含 `main()` 函数、主菜单，并协调对其他模块的调用。

任务书中的需求天然地可以划分为几个不同类别（项目、学生、报名、统计）。基于这些类别进行模块化设计，有利于实现“关注点分离”。例如，`event_management.c` 应该只关心项目数据，而不必了解学生数据是如何存储的。这种做法降低了每个模块内部的复杂性。每个模块将拥有其头文件，声明其公共函数及可能定义的特定数据类型（尽管核心结构体将是全局可访问的，可能定义在一个中央的 `structures.h` 或类似文件中，并被所有模块包含）。

表2: 功能-模块映射表

PDF功能需求	建议模块	示例函数名
运动项目管理		
显示所有运动项目信息	event_management.c	display_all_events()
添加运动项目信息	event_management.c	add_sport_event()
删除运动项目信息	event_management.c	delete_sport_event()
修改运动项目信息	event_management.c	modify_sport_event()
查找运动项目信息	event_management.c	search_sport_event()
学生信息管理		
显示所有学生信息	student_management.c	display_all_students()
添加学生信息	student_management.c	add_student_info()
删除学生信息	student_management.c	delete_student_info()
修改学生信息	student_management.c	modify_student_info()
查找学生信息	student_management.c	search_student_info()
项目报名管理		
学生报名	registration_management.c	register_student_for_event()
取消报名	registration_management.c	cancel_registration()
显示所有报名信息	registration_management.c	display_all_registrations()
查找报名信息	registration_management.c	search_registration_info()
录入成绩	registration_management.c	enter_score()

PDF功能需求	建议模块	示例函数名
统计查询功能		
统计每个项目的报名人数	<code>statistics_reporting.c</code>	<code>stats_participants_per_event()</code>
统计每个学院的报名人数 (总计 & 按项目)	<code>statistics_reporting.c</code>	<code>stats_participants_per_college()</code> , <code>stats_college_event_breakdown()</code>
统计每个学生的报名项目数	<code>statistics_reporting.c</code>	<code>stats_events_per_student()</code>
查询比赛成绩 (含排名)	<code>statistics_reporting.c</code>	<code>query_scores_and_rankings()</code>
保存统计信息到文件	<code>statistics_reporting.c</code>	<code>save_stats_to_file()</code> (可调用其他统计函数)
其他功能		
登录、退出系统	<code>main.c</code> / <code>utils.c</code>	<code>login_system()</code> , <code>exit_system()</code> (主循环控制)

此表格在1中的抽象需求与将要编写的具体C代码之间架起了一座桥梁。通过将每个功能映射到特定模块并建议函数名，它为项目提供了一个清晰的组织结构。这直接满足了“模块化设计”标准 1，并通过将整个系统编码的庞大任务分解为更小、定义明确的部分，使其更易于管理。它还有助于规划模块之间的依赖关系。

D. 保证持久性：数据存储与检索策略 (文件I/O)

任务书中明确提到了“保存统计信息到文件”¹。然而，一个实用的系统需要使其核心数据（项目、学生、报名信息）在程序会话之间得以保留，而不仅仅是统计数据。尽管¹并未强制要求为所有数据实现保存/加载功能，但这无疑是一个高度实用的特性，并且符合创建一个具有“实用性”系统的目标。若无此机制，程序退出时所有数据都将丢失。因此，设计中应包含在程序启动时从文件加载数据，以及在退出前或用户指令下将数据保存到文件的机制。

- **文件格式选择：**
 - **CSV (逗号分隔值)：**人类可读，易于解析，适合文本型数据。每种结构体类型可对应一个CSV文件（例如 `events.csv`, `students.csv`, `registrations.csv`）。
 - **二进制文件：**更紧凑，潜在的I/O速度更快，但非人类可读。可以直接读写 `struct` 实例（但需注意指针和内存对齐问题）。

- **建议：**对于此项目，**推荐使用CSV文件**，因其简单且易于调试（可用文本编辑器或电子表格软件打开）。
- 在 `file_io.c` / `file_io.h` 中实现：
 - `load_events_from_file(const char* filename, SportEvent** events_array, int* count)`
 - `save_events_to_file(const char* filename, const SportEvent* events_array, int count)`
 - 为 `StudentInfo` 和 `RegistrationInfo` 实现类似的函数。
 - `statistics_reporting.c` 中的函数将负责处理统计输出文件的特定格式。

II. 分阶段实施指南：逐步构建C应用程序

本节将指导完成每个模块的编码。对于每个函数，需考虑其参数、返回值、核心逻辑以及与其他模块或数据的交互。

A. 基础元素：实现核心数据结构和工具函数

首先建立基本的数据结构和通用工具函数，可以为后续开发提供一个稳定的基础。试图在没有这些构建块的情况下编写复杂逻辑，会导致代码重复和错误。

- **1. 定义结构体：**创建一个头文件（例如 `datamodel.h` 或 `types.h`）来声明在表1中定义的 `SportEvent`，`StudentInfo`，和 `RegistrationInfo` 结构体。这确保所有模块使用相同的定义。
- **2. 全局数据存储**
 - ：确定如何存储这些结构体的集合。
 - 示例： `SportEvent* g_sport_events = NULL; int g_event_count = 0; int g_event_capacity = 0;`（学生和报名信息类似）。这些通常是全局变量，定义在 `main.c` 或专门的数据管理C文件中，并在一个全局头文件中声明为 `extern`。
 - 实现动态数组管理：提供添加元素（若 `count == capacity` 则重新分配内存）和移除元素的函数。
- **3. `utils.c` / `utils.h`**
 - ：
 - **输入函数：**安全的函数，用于读取字符串（防止缓冲区溢出）、整数等。例如 `get_string_input(char* buffer, int size)`。
 - **验证函数：** `is_valid_event_id_format()`，`is_valid_student_id_format()`，`is_date_valid()`。
 - **ID生成 (可选创新)：** `generate_unique_registration_id()` 用于创建唯一ID，而非手动输入。
 - **字符串修剪：**移除输入字符串首尾的空白字符。

B. 模块1： `event_management.c` / `event_management.h` (运动项目管理)

参考 1 和 1 获取功能列表。

- `add_sport_event()`
- ：

- 提示用户输入所有项目详情（依据 `SportEvent` 结构体）。
- 验证输入（例如：报名上限 > 0 ，项目编号唯一）。
- 为新项目分配内存，填充数据，并添加到全局项目数组/列表中。
- 设置 `current_registrations` 为0。
- `display_all_events()`
 - :
 - 遍历项目数组/列表。
 - 格式化打印每个项目的详细信息。
 - 11 指明“按照项目类型显示”。这意味着在显示前要么按类型排序数据，要么迭代并分组。排序通常更清晰。
- `search_sport_event()`
 - :
 - 提示用户输入搜索条件（ID、名称等，依据 1）。
 - 迭代查找匹配的项目并显示其详细信息。
- `modify_sport_event()`
 - :
 - 提示输入要修改的项目ID。
 - 查找项目。若未找到，显示错误。
 - 显示当前详情。提示用户选择要更改的字段及新值。验证并更新。
- `delete_sport_event()`
 - :
 - 提示输入要删除的项目ID。
 - 查找项目。若未找到，显示错误。
 - 删除项目时，需要考虑其对现有报名信息的影响。基础需求并未指明如何处理这种情况。一个健壮的系统可能会阻止删除尚有有效报名的项目，或提供选项来同时取消这些报名。目前，可先实现简单删除，并将此标记为潜在的改进/创新点（例如，检查 `RegistrationInfo` 数据）。
 - 从数组/列表中移除（移动元素或重新链接）。

C. 模块2: `student_management.c` / `student_management.h` (学生信息管理)

参考 1 和 1 获取功能列表。其结构与项目管理类似：`add_student_info()`，`display_all_students()`，`search_student_info()`，`modify_student_info()`，`delete_student_info()`。

- 关键验证：学号唯一。
- 与项目删除类似，删除一个已存在有效报名的学生信息时，也需要审慎处理。是否应自动取消其所有报名？`delete_student_info()` 函数理想情况下应与报名数据交互以确保数据一致性，这是构建稳健系统的一个重要细节。

D. 模块3: `registration_management.c` / `registration_management.h` (项目报名管理)

参考 1 和 1 获取功能列表。

- `register_student_for_event()`
:
 - 提示输入学号和项目编号。
 - 验证: 学生是否存在? 项目是否存在? 项目是否处于“报名中”状态? 项目的 `current_registrations < registration_limit` 是否成立? 该学生是否已报名此项目?
 - 若所有检查通过:
 - 生成 `registration_id` (例如: "REG" + 序列号, 或基于时间戳)。
 - 记录 `registration_time`。
 - 添加到报名数组/列表。
 - 增加对应 `SportEvent` 的 `current_registrations`。
- `cancel_registration()`
:
 - 提示输入报名编号或学号+项目编号。
 - 查找报名记录。若未找到, 报错。
 - 从报名数组/列表中移除。
 - 减少对应 `SportEvent` 的 `current_registrations`。
- `display_all_registrations()`
:
 - 遍历报名记录。
 - 11 指明“按照报名时间, 逐个显示”。这意味着在显示前需按 `registration_time` 排序。
 - 为提高可读性, 显示学生姓名和项目名称, 而非仅仅ID (这需从学生/项目数据中查找)。
- `search_registration_info()`
:
 - 提示输入条件 (报名ID、学号、项目ID) 。查找并显示。
- `enter_score()`
:
 - 提示输入报名编号或学号+项目编号。
 - 查找报名记录。
 - 提示输入成绩。更新 `score` 字段。
 - 可以考虑确保项目状态为“已结束”才允许录入成绩 (任务书未明确, 但前者更符合逻辑)。

E. 模块4: `statistics_reporting.c` / `statistics_reporting.h` (统计查询功能)

参考 1 和 1 获取功能列表。这些函数将涉及遍历数据数组/列表、筛选和计数。

- `stats_participants_per_event()`: 遍历 `SportEvent` 数组, 显示 `event_name` 和 `current_registrations`。
- `stats_participants_per_college()`
 - :
 - 需要遍历 `RegistrationInfo`。对每条报名记录, 获取 `student_id`, 然后从 `StudentInfo` 中找到该学生的 `college`。
 - 按学院汇总报名人数。
 - 对于“每个学院每个项目的报名人数”: 这是一个更复杂的嵌套聚合。
- `stats_events_per_student()`: 遍历 `StudentInfo`。对每个学生, 统计其在 `RegistrationInfo` 中的条目数。
- `query_scores_and_rankings()`
 - :
 - 提示输入项目ID或学号。
 - 若为项目ID: 查找该项目所有非空成绩的报名记录。按成绩排序 (计时项目升序, 计分项目降序——需定义成绩如何用于排名)。显示排名列表。
 - 若为学号: 查找该学生所有带成绩的报名记录。显示。
- `save_stats_to_file(const char* filename, int choice_of_stat)`
 - :
 - 此函数可接受一个参数, 指明要保存哪项统计数据。
 - 它将调用相关的内部统计生成函数, 然后格式化输出并写入指定文件。

表3: 统计查询功能概览

查询描述	输入参数 (若有)	输出格式 / 显示的关键信息	涉及的核心数据结构
统计每个项目的报名人数	无	项目名称, 已报名人数	<code>SportEvent</code> (使用 <code>current_registrations</code>)
统计每个学院的报名人数 (总计)	无	学院名称, 该学院总报名人数	<code>RegistrationInfo</code> , <code>StudentInfo</code> (获取学院信息)
统计每个学院每个项目的报名人数	无	学院名称, 项目名称, 该学院在该项目的报名人数	<code>RegistrationInfo</code> , <code>StudentInfo</code> , <code>SportEvent</code>
统计每个学生的报名项目数	无	学号, 学生姓名, 已报名项目数量	<code>RegistrationInfo</code> , <code>StudentInfo</code>

查询描述	输入参数 (若有)	输出格式 / 显示的关键信息	涉及的核心数据结构
查询比赛成绩 (按项目ID, 含排名)	项目ID	学生姓名, 成绩, 排名	<code>RegistrationInfo</code> , <code>StudentInfo</code> (获取姓名), <code>SportEvent</code> (获取项目背景)
查询比赛成绩 (按学号)	学号	项目名称, 成绩	<code>RegistrationInfo</code> , <code>SportEvent</code> (获取项目名称)

统计查询是系统功能的重要组成部分。此表分解了每个查询，指明了其输入、预期输出以及处理所需的数据。这种清晰度对于设计 `statistics_reporting.c` 中的逻辑至关重要，有助于确保系统地满足 11 中每项查询要求的所有方面。

F. 核心系统操作：实现登录、退出及主用户界面 (`main.c`)

- 登录 (基础版)
：
 - 11 列出了“登录、退出系统”。
 - 对于C控制台项目，除非作为创新点特别指出，否则完整的用户认证系统可能过于复杂。
 - 一个简单的方法是：显示欢迎信息。如果需要更高级的功能，可以设置预定义的用户名/密码（硬编码或从简单配置文件读取）。
 - 虽然简单的登录对于满足基本要求已足够，但一个更复杂的系统可能会引入不同的用户角色（如管理员、普通学生）。如果时间允许，这是一个潜在的创新领域。目前，重点应放在实现一个功能性的入口点。
- 主菜单
：
 - 提供选项：
 1. 运动项目管理 -> 子菜单
 2. 学生信息管理 -> 子菜单
 3. 项目报名管理 -> 子菜单
 4. 统计查询功能 -> 子菜单
 5. 保存数据 (将所有数据保存到文件 - 创新点，但属良好实践)
 6. 退出系统 -> 在退出前调用保存数据的函数。
- 子菜单：每个主选项会引导至另一个菜单，列出该模块内的具体功能（例如：添加项目、删除项目）。
- 用户交互循环：使用 `do-while` 或 `while` 循环配合 `switch-case` 处理菜单选择。
- 清屏：使用 `system("cls")` (Windows) 或 `system("clear")` (Linux/macOS) 以改善操作间的用户界面呈现效果，或者如果需要，可以实现一种平台无关的清屏方式。

III. 打造优质代码：最佳实践与验证

本节聚焦于对提交高质量成果至关重要的方面，参考了1中的评分标准。

A. 稳健性：必要的错误处理和输入验证技术

- **永不信任用户输入：**验证所有输入的类型、范围、格式及逻辑一致性（例如，不能报名一个已满额的项目）。
- **函数返回值：**可能失败的函数（例如文件I/O、搜索未找到）应返回状态码（例如0表示成功，-1表示错误）或特定的错误指示符。调用者必须检查这些返回值。
- **指针检查：**始终检查 `malloc`, `calloc`, `realloc` 是否返回 `NULL`。
- **防御性编程：**处理边界情况（例如空数据文件、尚未添加任何项目/学生的情况）。
- **清晰的错误消息：**告知用户发生了什么错误，以及可能如何修正。

B. 专业性：遵循C语言编码规范

直接参考 11 中的“代码规范性要求”：

1. **命名：**函数、变量等命名应见名知意。使用 `snake_case_for_variables_and_functions` (小写字母加下划线分隔单词的变量和函数名), `UPPER_SNAKE_CASE_FOR_CONSTANTS` (大写字母加下划线分隔单词的符号常量名), 这符合1中“用小写字母为变量、函数命名,用大写字母为符号常量命名”的要求。

2. **每行一条语句。**

3. 花括号

:

左右花括号各占一行,且上下对齐

。

C

```
if (condition)
{
    // statement
}
```

4. **缩进：**整个程序采用逐层缩进的方式进行书写 (使用一致的缩进，例如4个空格)。

5. 注释

:

对关键代码进行注释,提高程序的可读性和可维护性

。

- 函数头部注释应解释其用途、参数、返回值。

表4: C语言编码规范核查表

规范 (第11页)	符合性检查
见名知意的命名 (变量/函数小写, 常量大写)	是/否 - 审查变量/函数名。
每行一条语句	是/否 - 扫描代码, 检查单行多语句的情况。

规范 (第11页)	符合性检查
花括号各占一行，上下对齐	是/否 - 检查 <code>if</code> , <code>for</code> , <code>while</code> 及函数的花括号。
一致的缩进	是/否 - 目视检查或使用代码格式化工具。
关键代码/函数注释	是/否 - 审查关键代码段和函数定义是否有注释。

11 明确列出了将要评分的编码标准。此核查表提供了一种在提交前对照这些具体要求自我评估代码的快捷方式。遵守这些标准不仅能提高分数，还能使代码更具可读性和可维护性，这些都是任何程序员的宝贵技能。

C. 自信度：单元测试与集成测试策略

- 单元测试
 - ：测试单个函数，尤其是在工具和管理模块中的函数。
 - 示例：对于 `add_sport_event`，测试添加有效项目、添加ID已存在的项目（应失败）、添加数据无效的项目。
- 集成测试
 - ：测试模块如何协同工作。
 - 示例：学生报名参加一个项目。然后，尝试删除该项目——系统如何响应？检查 `current_registrations` 是否正确更新。
- 测试用例
 - ：如实验报告模板上下文所述
 - 1
 - ，考虑：
 - 正常情况（预期输入）。
 - 边界情况（例如：报名人数达到上限，空输入）。
 - 错误情况（无效输入，不存在的ID）。
- 手动测试：通过控制台用户界面，从用户角度系统地检查所有功能。

IV. 提升项目：潜在的增强功能与创新点

参考 11：“...需要自行设计更具有实用性和创新性的系统功能。”以及对“难度及创新”的评分 1。

A. 超越基本要求的创意构想

- **增强型登录**：基础的角色区分（管理员 vs. 普通用户），赋予不同权限（例如，只有管理员可以添加/删除项目）。
- **高级搜索/筛选**：更复杂的搜索条件（例如，特定时间范围内的项目，来自特定学院和班级的学生）。
- **数据导入/导出**：允许从预定义的CSV格式导入学生名单或项目日程。
- **自动ID生成**：为 `event_id`, `student_id` (如果允许)，特别是 `registration_id` 实现自动生成。
- **冲突检查**：防止学生报名参加两个比赛时间重叠的项目。

- **审计日志**：将重要操作（如添加项目、删除学生、完成报名）的基本日志记录到文件，以供追溯。
- **更完善的排名机制**：处理成绩相同的情况，为不同类型的项目设置不同的排名标准。
- **数据验证的稳健性**：更全面的检查，或许可以对ID格式或联系方式使用正则表达式。
- **用户友好的输入**：对于日期等输入，可以引导用户或使用简单的日期验证。

B. 对未来可扩展性或功能增加的考量

尽管不要求完全实现，但思考设计如何适应未来的变化能体现出前瞻性。例如，如果要添加用户角色，当前模块间的交互会如何改变？保持模块间的松耦合（低依赖性）对此有所帮助。

本详细指南提供了一个全面的路线图。请记住，要增量实施，频繁测试，并时常回顾1中的要求，以确保项目按计划进行。